

Security Audit

Indie.fun 3rd (Gaming)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	18
Conclusion	19
Our Methodology	20
Disclaimers	22
About Hashlock	23

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Indie.fun team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Indie.fun is a platform for indie game developers and enthusiasts, launched in January 2025. The project aims to provide a space for indie game creators to showcase their work, connect with players, and potentially monetize their creations.

Simply put, Indie.fun is a hub where indie game developers can submit their games, players can discover unique gaming experiences, and the indie gaming community can thrive through direct connections between creators and fans.

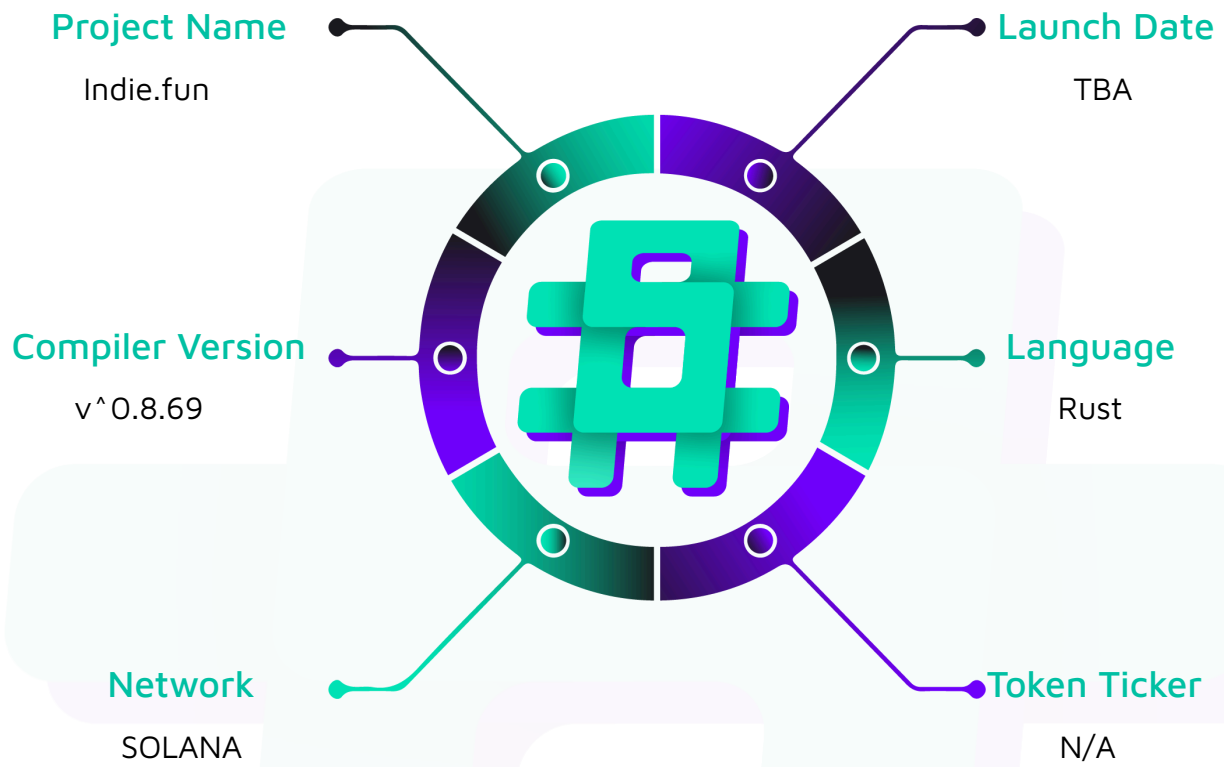
Project Name: Indie.fun

Compiler Version: v^0.8.69

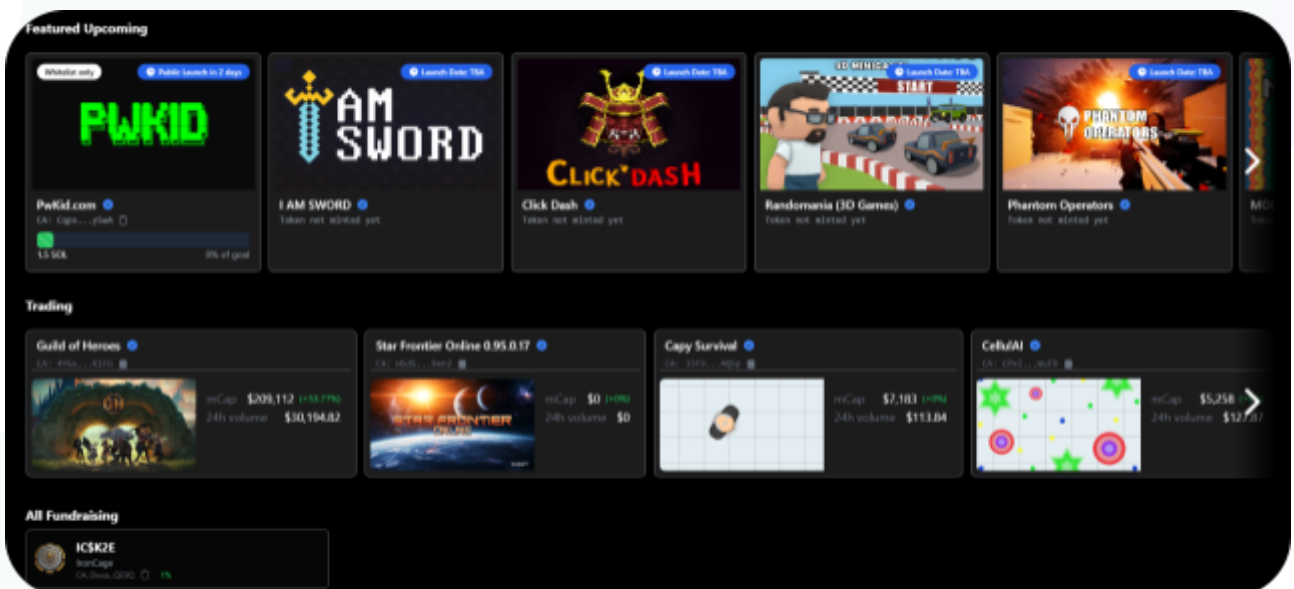
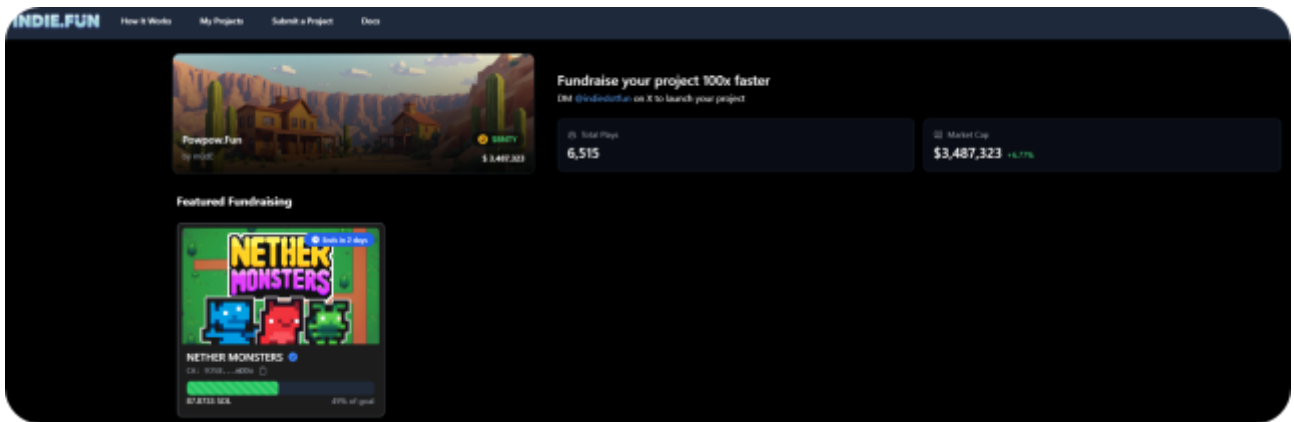
Website: <https://indie.fun/>

Logo:

INDIE.FUN

Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the Rust code within the Indie.fun project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Indie.fun Protocol Smart Contracts
Platform	Rust
Audit Date	February, 2025
Contract 1	campaign.rs
Contract 1 MD5 Hash	e886c7572a465590137ad159874140c7
Contract 2	proxy_initialize.rs
Contract 2 MD5 Hash	e1a3b0f1bb366b4f02f6c53bb5dbd871
Contract 3	collect_fees.rs
Contract 3 MD5 Hash	f6f015e5995e952ca10be963d72320a7
Contract 4	lock_liquidity.rs
Contract 4 MD5 Hash	7401f0078203ffe6ef698cf9eb130dd1
Contract 5	back_campaign.rs
Contract 5 MD5 Hash	9511f23f503d806ad2c49f4188a1ea2d
GitHub Commit Hash	e7fc786f85bbb1bb8be7d013d153c0c8025f58a1

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. We initially identified some vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
campaign.rs Allows users to: - Create a token and start a crowdfunding campaign (to launch a game later on using the token as in-game currency).	Contract achieves this functionality.
proxy_initialize.rs Allows to: - verify that the campaign has reached its fundraising goal, then distributes SOL and tokens from the campaign vault to both the campaign creator and treasury according to preset fee structures. It handles wrapping SOL into WSOL when required and synchronizes balances before calling an external CPI to Raydium's liquidity pool program to initialize the pool with the correct token amounts. Finally, based on the LP tokens handling configuration, it either burns the resulting LP tokens or transfers them to the campaign creator, cleans up temporary accounts, and updates the campaign's status to indicate that the liquidity pool has been created.	Contract achieves this functionality.
back_campaign.rs Allows users to:	Contract achieves this functionality.

- Contribute funds (SOL) to back an active crowdfunding campaign before its end time. - Update the campaign's total raised amount and backer records while enforcing presale contribution limits. - Automatically allocate in-game tokens to backers based on their proportional contribution relative to the campaign's fundraising goal. - Deduct and transfer a 1.5% treasury fee from each contribution, while routing the remaining funds to the campaign vault.	
collect_fees.rs Allows authorized signers to: - collect liquidity pool fees after the campaign's liquidity is locked, splitting them between the campaign creator and treasury based on predefined percentages. It validates the campaign status and fee configuration before executing token transfers to the respective accounts.	Contract achieves this functionality.
lock_liquidity.rs Allows to: - handle the process of securely locking a campaign's liquidity tokens once the liquidity pool is created. It first confirms that the campaign is in the correct state (i.e., liquidity pool created and LP token handling set to "BurnAndEarnTradeFees"). The module then	Contract achieves this functionality.

gathers all LP tokens from the campaign vault, constructs the necessary program-derived addresses for proper authorization, and invokes a CPI call to Raydium's liquidity locking program to lock these tokens—minting a fee NFT in the process.	
---	--

Code Quality

This audit scope involves the smart contracts of the Indie.fun project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Indie.fun project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] back_campaign.rs - Overflow error is set in multiple places instead of underflow error

Description

There are currently many instances where there is an overflow error set instead of the underflow error as the overflow is impossible in this case. A few examples:

```
let treasury_amount = (amount as u128)

    .checked_mul(15)

    .ok_or(ModdFunError::OverflowError)?

    .checked_div(1000)

    .ok_or(ModdFunError::OverflowError)? as u64;
```

The operation here is multiplication by 15 and then division by 1000 so the only possible error is an underflow.

```
let treasury_amount = total_amount

    .checked_sub(creator_amount)

    .ok_or(ModdFunError::OverflowError)?; // Give rounding advantage to creator
```

The same scenario happens here - the operation is subtraction so the only possible error is an underflow and not overflow.

Recommendation

Change the naming so the errors are correctly named everywhere.

Status

Resolved

[L-02] backerinfo.rs - Missing bump parameter when defining a PDA account

Description

At the moment, there is no bump parameter set when defining the BackerInfo struct. This leads to a situation where PDA addresses can be derived properly as there is no necessary bump defined.

Vulnerability Details

Let's take a look at how the BackerInfo struct is initialized:

```
#[account]

#[derive(InitSpace)]

pub struct BackerInfo {

    pub backer: Pubkey,

    pub campaign: Pubkey,

    pub amount: u64,

}
```

However, the BackerInfo is a PDA address:

```
#[account(

    init_if_needed,

    payer = backer,

    space = DISCRIMINATOR_SIZE + BackerInfo::INIT_SPACE,

    seeds = [b"backer", campaign.key().as_ref(), backer.key().as_ref()],

    bump,

)]

pub backer_info: Account<'info, BackerInfo>
```

According to the Anchor documentation:

PDA's are addresses that are derived deterministically using a combination of user-defined seeds, a bump, and a program's ID. However, in the current functionality, the canonical bump (that's used to derive an address) can't be stored in the `backerinfo` struct.

Impact

PDA derivation will result in errors.

Recommendation

Add a bump parameter in the struct.

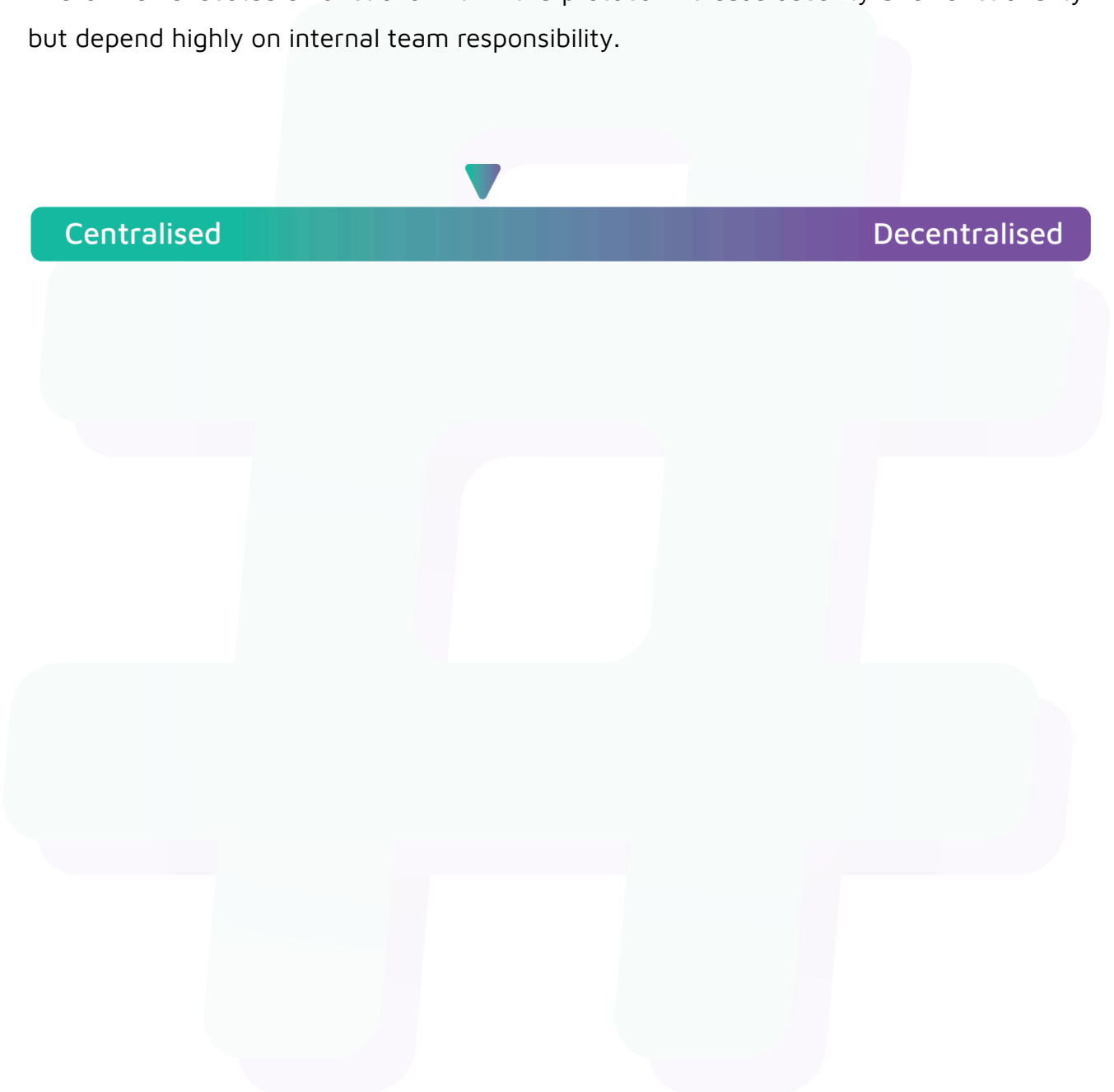
Status

Resolved

Centralisation

The Indie.fun project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Indie.fun project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.